← Danh sách

0/30 câu đã làm · 0/28 câu chấm máy đúng lần đầu

Làm lại từ đầu

CĐ-05 · Lập trình Python

Luyện kiến thức: không tính giờ, chọn xong là biết đúng/sai và xem giải thích. Tiến độ lưu riêng với Luyện đề.

Mã đề **PETER-CD05-6969** · Người làm: Peter: gửi mã này cho mentor khi chữa bài.

Phần A — Nhận biết

Câu 1 · T05-A04 · 3 điểm · Trắc nghiệm · Nhận biết

Nhóm nào dưới đây gồm **toàn** kiểu dữ liệu bất biến (immutable) trong Python?

- ○ **A.** `int`, `str`, `tuple`, `frozenset`
- ○ **B.** `float`, `str`, `dict`, `bytes`
- ○ **C.** `str`, `tuple`, `set`, `bool`
- ○ **D.** `int`, `str`, `list`, `tuple`

Câu 2 · T05-A03 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Dashboard giám sát API in tổng số request và tỉ lệ lỗi bằng f-string:

```
n = 1234567
r = 0.0456
print(f"{n:,} | {r:.1%}")
```

Kết quả in ra là gì?

- ○ **A.** `1,234,567 | 0.0%`
- ○ **B.** `1,234,567 | 4.6%`
- ○ **C.** `1.234.567 | 4.6%`
- ○ **D.** `1,234,567 | 4.5%`

Câu 3 · T05-A07 · 3 điểm · Trắc nghiệm · Nhận biết

Với các tham số mặc định, decorator `@dataclass` tự sinh những phương thức nào cho lớp?

- ○ **A.** `__init__`, `__repr__` và `__eq__`
- ○ **B.** Chỉ `__init__`
- ○ **C.** `__init__`, `__hash__` và `__str__`
- ○ **D.** `__init__`, `__repr__` và `__lt__`

Câu 4 · T05-A05 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

File log có hàng triệu dòng, `lines` là đối tượng file đang mở. Bạn muốn tính độ dài từng dòng **lần lượt** mà không tạo sẵn toàn bộ kết quả trong bộ nhớ. Biểu thức nào tạo ra một **generator**?

- ○ **A.** `(len(l) for l in lines)`
- ○ **B.** `tuple(len(l) for l in lines)`
- ○ **C.** `{len(l) for l in lines}`
- ○ **D.** `[len(l) for l in lines]`

Câu 5 · T05-A02 · 3 điểm · Trắc nghiệm · Nhận biết

Lệnh sau in ra gì?

```
print(7 // 2, 7 / 2, -7 // 2)
```

- ○ **A.** `3 3 -4`
- ○ **B.** `3 3.5 -4`
- ○ **C.** `3.5 3.5 -3.5`
- ○ **D.** `3 3.5 -3`

Câu 6 · T05-A01 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Bạn muốn đếm số lần xuất hiện của từng cặp (endpoint, mã trạng thái) trong log truy cập bằng một `dict`. Giá trị nào dưới đây dùng được làm **khóa** của `dict`?

- ○ **A.** `{"/api/orders", 503}`
- ○ **B.** `{"/api/orders": 503}`
- ○ **C.** `["/api/orders", 503]`
- ○ **D.** `("/api/orders", 503)`

Câu 7 · T05-A08 · 3,5 điểm · Điền kết quả · Nhận biết · tình huống

Một bản ghi log được cập nhật trong quá trình xử lý. Đoạn code sau in ra gì? Ghi đúng như Python in.

```
rec = {"level": "ERROR", "code": 503}
rec["retry"] = 2
rec["code"] = 500
print(len(rec), list(rec))
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 8 · T05-A06 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Một dòng log đã parse thành `rec = {"ts": "08:15:00", "msg": "timeout"}`. Câu lệnh `rec["status"]` ném ra ngoại lệ gì?

- ○ **A.** `AttributeError`
- ○ **B.** `IndexError`
- ○ **C.** Không ném lỗi, trả về `None`
- ○ **D.** `KeyError`

Phần B — Thông hiểu

Câu 9 · T05-B04 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Trước khi ghi đè, bạn sao chép một bản ghi để thử nghiệm:

```
rec = {"id": 7, "tags": ["api"]}
cp = rec.copy()
cp["id"] = 8
cp["tags"].append("retry")
print(rec)
```

Kết quả in ra là gì?

- ○ **A.** `{'id': 7, 'tags': ['api']}`
- ○ **B.** `{'id': 8, 'tags': ['api', 'retry']}`
- ○ **C.** `{'id': 8, 'tags': ['api']}`
- ○ **D.** `{'id': 7, 'tags': ['api', 'retry']}`

Câu 10 · T05-B01 · 3 điểm · Trắc nghiệm · Thông hiểu

Đoạn code sau in ra gì?

```
buf = [0, 1, 2, 3, 4, 5]
buf[1:3] = ["x"]
print(buf)
```

- ○ **A.** Báo `ValueError`
- ○ **B.** `[0, 'x', 2, 3, 4, 5]`
- ○ **C.** `[0, 'x', 3, 4, 5]`
- ○ **D.** `[0, ['x'], 3, 4, 5]`

Câu 11 · T05-B06 · 3,5 điểm · Điền kết quả · Thông hiểu · tình huống

Đếm tần suất từ khóa trong log bằng dict comprehension. Đoạn code sau in ra gì? Ghi đúng như Python in.

```
words = "Error timeout error retry ERROR ok".lower().split()
freq = {w: words.count(w) for w in words}
```

```
print(freq)
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 12 · T05-B03 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Client gọi API ghép cấu hình mặc định với cấu hình người dùng (Python 3.9+):

```
defaults = {"timeout": 5, "retries": 3}
user = {"retries": 5, "verbose": True}
cfg = defaults | user
print(cfg["retries"], len(cfg), defaults["retries"])
```

Kết quả in ra là gì?

- ○ **A.** `5 4 3`
- ○ **B.** `5 3 3`
- ○ **C.** `3 3 3`
- ○ **D.** `5 3 5`

Câu 13 · T05-B10 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Trích mã trạng thái và độ trễ từ log:

```
import re
log = "GET /api/users 200 12ms; GET /api/orders 503 480ms"
print(re.findall(r"(\d{3}) (\d+)ms", log))
```

Kết quả in ra là gì?

- ○ **A.** `[('200', '12')]`
- ○ **B.** `['200 12ms', '503 480ms']`
- ○ **C.** `[(200, 12), (503, 480)]`

- **D.** `[('200', '12'), ('503', '480')]`

Câu 14 · T05-B02 · 3,5 điểm · Điền kết quả · Thông hiểu · tình huống

Tách một dòng access log bằng unpacking có dấu sao. Đoạn code sau in ra gì?

```
first, *mid, last = "GET /api/v1/users 200 35ms".split()
print(first, mid, last)
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 15 · T05-B07 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Đoạn code tính tổng độ dài các dòng log:

```
lines = ["GET /a", "POST /b", "GET /c"]
sizes = (len(l) for l in lines)
print(sum(sizes), sum(sizes))
```

Kết quả in ra là gì?

- **A.** Báo lỗi `StopIteration`
- **B.** `19 19`
- **C.** `19 0`
- **D.** Báo lỗi `TypeError`

Câu 16 · T05-B08 · 3 điểm · Trắc nghiệm · Thông hiểu

Đoạn code sau in ra gì? (Dấu ⏎ phân tách các dòng in ra.)

```
def fetch():
    try:
        print("try")
        return "data"
    finally:
```

```
        print("cleanup")

print(fetch())
```

- ○ **A.** `try` ⏎ `cleanup` ⏎ `data`
- ○ **B.** `try` ⏎ `cleanup` ⏎ `None`
- ○ **C.** `try` ⏎ `data` ⏎ `cleanup`
- ○ **D.** `try` ⏎ `data`

Câu 17 · T05-B05 · 3,5 điểm · Điền kết quả · Thông hiểu

Đoạn code sau in ra gì? Ghi đúng như Python in.

```
def normalize(xs):
    xs.append(0)
    xs = [9]
    xs.append(1)
    return xs

data = [5]
out = normalize(data)
print(data, out)
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 18 · T05-B09 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Một lớp gom dòng log của từng worker:

```
class Collector:
    lines = []
    def add(self, line):
        self.lines.append(line)

a = Collector()
b = Collector()
a.add("E1")
a.add("E2")
```

```
b.add("W1")
print(len(a.lines), len(b.lines))
```

Kết quả in ra là gì?

- ○ **A.** `0 0`
- ○ **B.** `2 1`
- ○ **C.** Báo `AttributeError`
- ○ **D.** `3 3`

Phần C — Vận dụng

Câu 19 · T05-C01 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Sau khi đếm từ trong log, bạn xếp hạng từ khóa: tần suất **giảm dần**, hoà thì theo **chữ cái tăng dần**.

```
freq = {"retry": 3, "error": 5, "ok": 3, "timeout": 5}
top = sorted(freq.items(), key=lambda kv: (-kv[1], kv[0]))
print([w for w, _ in top])
```

Kết quả in ra là gì?

- ○ **A.** `['timeout', 'error', 'retry', 'ok']`
- ○ **B.** `['ok', 'retry', 'error', 'timeout']`
- ○ **C.** `['error', 'timeout', 'ok', 'retry']`
- ○ **D.** `['error', 'timeout', 'retry', 'ok']`

Câu 20 · T05-C04 · 3,5 điểm · Điền kết quả · Vận dụng

Đoạn code sau dùng closure làm bộ đếm. Kết quả in ra là gì?

```
def make_counter():
    n = 0
    def inc(step=1):
        nonlocal n
        n += step
        return n
```

```
    return inc

c1 = make_counter()
c2 = make_counter()
c1()
c1(5)
c2()
print(c1(), c2(10))
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 21 · T05-C09 · 6 điểm · Code viết tay · Vận dụng · tình huống

Log của một hệ thống microservice đã được parse thành list các dict:

```
records = [
    {"service": "auth", "status": 200, "ms": 35},
    {"service": "pay",  "status": 503, "ms": 1200},
    {"service": "auth", "status": 500, "ms": 80},
    {"status": 200, "ms": 10},                       # thiếu service
    {"service": "pay",  "status": 200, "ms": 240},
    {"service": "auth", "status": 200, "ms": 41},
]
```

Viết hàm `summarize(records)` (chỉ dùng thư viện chuẩn) trả về `dict` gom nhóm theo `service`, mỗi giá trị là dict `{"n": số bản ghi, "err": số bản ghi có status >= 500, "max_ms": ms lớn nhất}`. Yêu cầu:

- bỏ qua bản ghi thiếu khóa `service` hoặc `status`; bản ghi thiếu `ms` thì coi `ms = 0`;
- thứ tự khóa của kết quả theo lần xuất hiện đầu tiên của service;
- không sửa `records` đầu vào.

Với dữ liệu trên, `summarize(records)` phải trả về:

```
{'auth': {'n': 3, 'err': 1, 'max_ms': 80}, 'pay': {'n': 2, 'err': 1, 'max_ms': 1200}}
```

Viết code Python như trên giấy (không chạy được)…

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.

Câu 22 · T05-C07 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Trích số từ một dòng log cấu hình:

```
import re
s = "user=42 retry=3"
a = re.match(r"\d+", s)
b = re.search(r"\d+", s)
print(a, b.group(), re.findall(r"=(\d+)", s))
```

Kết quả in ra là gì?

- A. `None 42 ['42', '3']`
- B. `None 42 ['42']`
- C. `None 42 ['=42', '=3']`
- D. `42 42 ['42', '3']`

Câu 23 · T05-C06 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Pipeline đọc log dùng generator. Đoạn code sau in ra gì? Ghi mỗi dòng output trên một dòng.

```
def read(lines):
    for l in lines:
        print("read", l)
        yield l

errs = (l for l in read(["ok", "E1", "E2"]) if l.startswith("E"))
print("start")
print(next(errs))
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 24 · T05-C08 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Hàng đợi tác vụ định nghĩa một dataclass:

```
from dataclasses import dataclass

@dataclass
class Job:
    name: str
    tags: list = []
```

Điều gì xảy ra khi Python thực thi định nghĩa lớp này?

- ○ **A.** Chạy bình thường, mọi `Job` dùng chung list
- ○ **B.** Báo lỗi khi tạo đối tượng `Job("a")`
- ○ **C.** Báo `ValueError` ngay khi định nghĩa lớp
- ○ **D.** Chạy bình thường, mỗi `Job` có list `tags` riêng

Câu 25 · T05-C03 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Bạn gom lỗi theo nhóm dịch vụ:

```
groups = dict.fromkeys(["web", "db"], [])
groups["web"].append("timeout")
print(groups)
```

Kết quả in ra là gì?

- ○ **A.** `{'web': ['timeout'], 'db': ['timeout']}`
- ○ **B.** `{'web': ['timeout'], 'db': None}`
- ○ **C.** Báo `TypeError`
- ○ **D.** `{'web': ['timeout'], 'db': []}`

Câu 26 · T05-C05 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Module retry tạo sẵn các hàm tính thời gian chờ (backoff) cho từng lần thử:

```
delays = [lambda: 2 ** k for k in range(4)]
print([d() for d in delays])
```

Kết quả in ra là gì?

- ○ **A.** `[1, 2, 4, 8]`
- ○ **B.** `[16, 16, 16, 16]`
- ○ **C.** `[8, 8, 8, 8]`
- ○ **D.** Báo `NameError`

Câu 27 · T05-C02 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Client gọi API đọc timeout từ biến toàn cục. Đoạn code sau in ra gì?

```
timeout = 5

def call(url, t=timeout):
    return f"{url}:{t}"

timeout = 30
print(call("a"), call("b", timeout))
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Phần D — Vận dụng cao

Câu 28 · T05-D01 · 3 điểm · Trắc nghiệm · Vận dụng cao · tình huống

Một lô request được lưu dạng tuple `(danh_sách_id, số_lần_thử)`:

```
batch = (["req-1"], 3)
try:
    batch[0] += ["req-2"]
except TypeError:
    print("TypeError")
print(batch)
```

Kết quả in ra là gì? (Dấu ⏎ phân tách các dòng in ra.)

- ○ **A.** `TypeError` ⏎ `(['req-1'], 3)`
- ○ **B.** `TypeError` ⏎ `(['req-1', ['req-2']], 3)`
- ○ **C.** `(['req-1', 'req-2'], 3)`
- ○ **D.** `TypeError` ⏎ `(['req-1', 'req-2'], 3)`

Câu 29 · T05-D03 · 6 điểm · Code viết tay · Vận dụng cao · tình huống

Gọi API thanh toán đôi khi lỗi mạng tạm thời. Viết decorator có tham số `retry(times=3, base=0.5, on=(ConnectionError, TimeoutError), sleep=time.sleep)` (chỉ dùng thư viện chuẩn) với đặc tả:

1. Gọi hàm được bọc tối đa `times` lần **tổng cộng**; thành công thì trả ngay kết quả.
2. Nếu lần thử thứ `i` (đếm từ 0) ném ngoại lệ thuộc `on` và vẫn còn lượt, gọi `sleep(base * 2**i)` rồi thử lại. **Không** ngủ sau lần thử cuối.
3. Hết lượt thì ném lại **chính** ngoại lệ cuối cùng (giữ nguyên kiểu và traceback).
4. Ngoại lệ không thuộc `on` (ví dụ `ValueError`) ném ra ngay, không thử lại.
5. Giữ `__name__` của hàm gốc; `times < 1` thì `retry(...)` ném `ValueError`.

Ví dụ (truyền `sleep=waits.append` để test không phải chờ thật):

```
waits = []
calls = {"n": 0}

@retry(times=4, base=0.5, sleep=waits.append)
def get_price(sku):
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("mất kết nối")
    return 42

print(get_price("A1"), calls["n"], waits)   # 42 3 [0.5, 1.0]
```

Viết code Python như trên giấy (không chạy được)…

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.

Câu 30 · T05-D02 · 3,5 điểm · Điền kết quả · Vận dụng cao

Đoạn code sau dùng dataclass bất biến có thứ tự. Kết quả in ra là gì?

```
from dataclasses import dataclass, field

@dataclass(order=True, frozen=True)
class Event:
    level: int
    msg: str = field(compare=False)

evs = [Event(2, "b"), Event(1, "a"), Event(2, "c")]
print(len(set(evs)), sorted(evs)[0].msg, Event(2, "x") == Event(2, "y"))
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra